

NAME:TANKISO MASOEBE

COMPUTER ARCHITECTURE WK3

1.SCENARIO ANALYSIS:

Pipelining is an important technique used in modern gaming consoles to improve performance and process tasks efficiently. For example, when a gaming console runs a game, the processor must perform several steps, such as fetching instructions, decoding them, executing calculations, accessing memory, and storing results. With pipelining, these steps can overlap instead of waiting for one instruction to finish completely before starting another. While one instruction is being executed, another can be decoded and a third can be fetched. This allows the processor to handle multiple instructions at different stages at the same time. As a result, the gaming console can process game instructions faster, reducing delays and improving responsiveness. Pipelining is especially useful in games that require fast calculations, such as racing, sports, and action games. Overall, pipelining helps the console use its CPU more efficiently, improving game performance, smoother gameplay, and faster response times.

2.CONCEPT RESEARCH:

Branch prediction is a technique used by modern computer processors to improve performance when handling instructions. When a program reaches a decision, such as an "if" statement, the processor may not immediately know which instruction should be executed next. Branch prediction allows the processor to predict which path the program is likely to take. If the prediction is correct, the processor continues working without waiting, saving time and improving speed. If the prediction is wrong, the processor removes the incorrectly processed instructions and follows the correct path. Branch prediction is important because modern CPUs use pipelining, where multiple instructions are processed at the same time. By reducing delays caused by decisions, branch prediction helps programs run faster and makes better use of the processor's resources.

3.TOOL PRACTICE:

```
#include <iostream>
```

```
#include <vector>
```

```
using namespace std;
```

```
int main() {
```

```
vector<string> instructions = {  
    "Instruction 1",  
    "Instruction 2",  
    "Instruction 3",  
    "Instruction 4"  
};
```

```
cout << "CPU Pipeline Simulation\n\n";
```

```
for (int cycle = 0; cycle < instructions.size() + 4; cycle++) {
```

```
    cout << "Cycle " << cycle + 1 << ": ";
```

```
    if (cycle >= 0 && cycle < instructions.size())
```

```
        cout << instructions[cycle] << " - Fetch, ";
```

```
    if (cycle >= 1 && cycle - 1 < instructions.size())
```

```
        cout << "Decode, ";
```

```
    if (cycle >= 2 && cycle - 2 < instructions.size())
```

```
        cout << "Execute, ";
```

```
    if (cycle >= 3 && cycle - 3 < instructions.size())
```

```
        cout << "Store";
```

```
        cout << endl;

    }

    return 0;

}
```

The C++ pipeline simulation helped me understand how a processor handles multiple instructions at different stages. I used Visual Studio Code to create a simple program that represents fetching, decoding, executing, and storing instructions. The simulation showed that instructions can overlap instead of waiting for the previous instruction to finish completely. This makes processing faster and improves CPU efficiency. Writing the program also helped me understand the importance of pipeline stages and how they work together. I learned that pipelining can increase performance by keeping different parts of the processor busy. Overall, this practical activity improved my understanding of computer architecture and CPU pipelining.

4.DIAGRAM:

CPU PIPELINE DIAGRAM



PIPELINE TIMING (CLOCK CYCLES)

Instruction	Cycle 1	Cycle 2	Cycle 3	Cycle 4	Cycle 5	Cycle 6	Cycle 7	Cycle 8
Instruction 1	Fetch	Decode	Execute	Memory	Write Back			
Instruction 2		Fetch	Decode	Execute	Memory	Write Back		
Instruction 3			Fetch	Decode	Execute	Memory	Write Back	
Instruction 4				Fetch	Decode	Execute	Memory	Write Back
Instruction 5					Fetch	Decode	Execute	Memory

Legend: Fetch Decode Execute Memory Write Back

This pipeline diagram shows how a CPU processes instructions through five stages: Fetch, Decode, Execute, Memory, and Write Back. Each instruction moves through these stages in order. The main advantage of pipelining is that different instructions can be processed at the same time in different stages. For example, while one instruction is being executed, another can be decoded and another can be fetched. This overlapping process allows the CPU to complete more instructions in less time. The diagram also shows how instructions move across different clock cycles. Overall, pipelining improves CPU performance, increases efficiency, and reduces the time needed to process multiple instructions.